

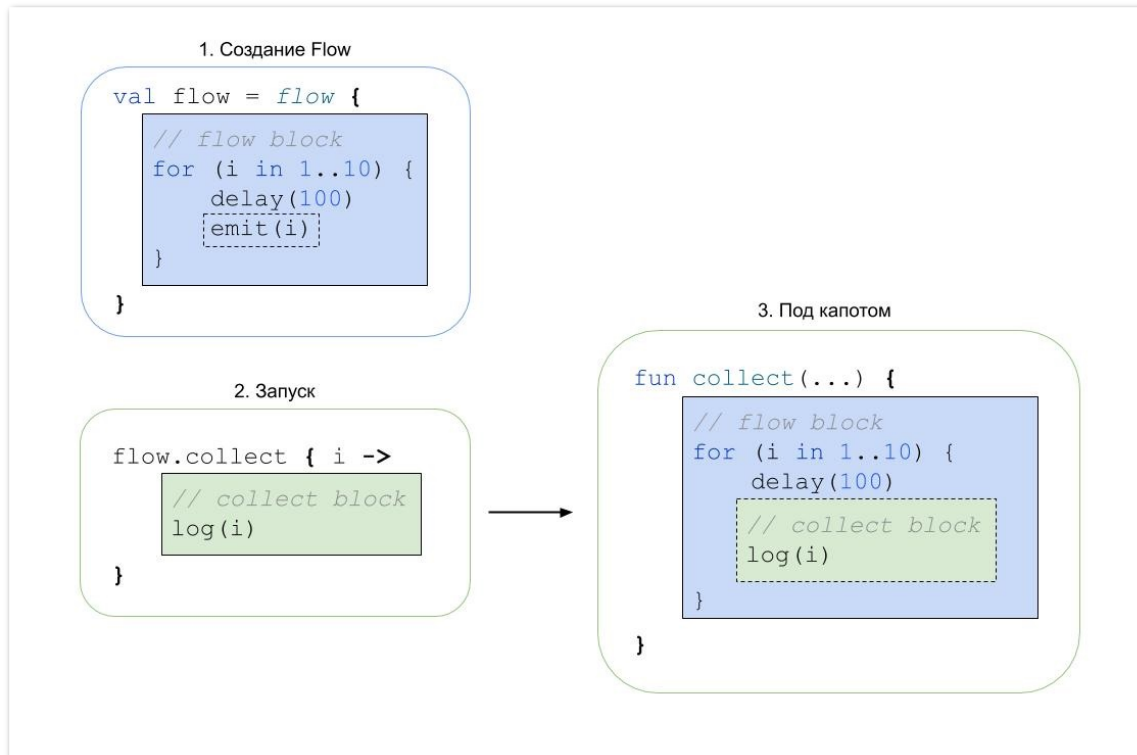
## Урок 21. Корутины. Flow. Операторы `channelFlow`, `flowOn`, `buffer`, `produceIn`.

04 сентября 2020

В этом уроке рассматриваем операторы `channelFlow`, `flowOn`, `buffer`, `produceIn`.

Во вводном уроке мы детально обсуждали, как под капотом работает Flow.

Напомню схему:



У нас есть 4 сущности:

- блок `flow`
- метод `emit()`
- метод `collect()`
- блок `collect`

Для дальнейшего понимания урока важно, чтобы вы различали их и понимали, кто кого и в каком порядке вызывает.

Получатель вызывает метод `collect()`, чтобы получить данные. Этот метод вызовет блок `flow`, который начнет создавать данные. Созданные данные отправляются в метод `emit()`, который переадресует их получателю в блок `collect`. Т.е. вся эта цепочка вызовов (`collect()` -> блок `flow` -> `emit()` -> блок `collect`) будет происходить последовательно в одной корутине, где был вызван метод `collect()`.

Хорошо бы иметь возможность сделать эту работу не такой последовательной. Например, вынести создание данных в отдельную корутину.

Пусть у нас есть Flow, который отправляет 3 элемента. Перед каждой отправкой он тратит секунду на то, чтобы создать данные.

```
1 val flow = flow {  
2     delay(1000)  
3     emit(1)  
4     delay(1000)  
5     emit(2)  
6     delay(1000)  
7     emit(3)  
8 }
```

Чтобы это работало параллельно, мы решаем раскидать код по отдельным корутинам:

```

1  val flow = flow {
2      coroutineScope {
3          launch {
4              delay(1000)
5              emit(1)
6          }
7
8          launch {
9              delay(1000)
10             emit(2)
11         }
12
13         launch {
14             delay(1000)
15             emit(3)
16         }
17     }
18 }

```

Напомним, что блок `flow` - это `suspend` функция. Мы не можем в ней напрямую стартовать корутины билдерами `launch`, `async`, `produce` и пр. Нам необходимо сначала вызвать `coroutineScope`. Он умеет из корутины, в которой вызвана `suspend` функция, извлекать `CoroutineScope`, который требуется для вызова билдеров корутин.

Теперь все данные будут подготовлены параллельно за одну секунду и отправлены нам.

Запускаем этот `Flow` методом `collect()` и получаем крэш:

```

java.lang.IllegalStateException: Flow invariant is violated:
  Emission from another coroutine is detected.
  Child of StandaloneCoroutine{Active}@ead1de6, expected child of StandaloneCoroutine{Active}@4b5ff27.
  FlowCollector is not thread-safe and concurrent emissions are prohibited.
  To mitigate this restriction please use 'channelFlow' builder instead of 'flow'

```

Так происходит потому, что `Flow` под капотом выполняет проверку, что методы `collect()` и `emit()` были выполнены в одной корутине. А в нашем случае мы вызов `emit()` выполняем из новой корутины. `Flow` определил это и сказал, что мы так не договаривались.

Давайте обсудим, почему так делать нельзя.

Во-первых - это не потокобезопасно. Мы из разных корутин (методом `emit`) вызываем один и тот же блок `collect`. Это может привести к параллельной работе с одним объектом внутри этого блока из разных потоков - так делать нельзя. Об этом мы еще поговорим в отдельном уроке.

Во-вторых - смена контекста. Метод `collect()` мы вызываем в корутине. Мы передаем туда блок кода `collect`, который будет обрабатывать получаемые данные, и предполагаем, что этот блок кода тоже будет выполнен в текущей корутине. И на самом деле так и будет, если все делать правильно и в блоке `flow` не запускать новые корутины. Потому что, как мы уже обсудили выше, метод `collect()` запустит блок `flow`, а он в свою очередь методом `emit()` запустит блок `collect`. И все это выполнится в корутине, где мы вызвали метод `collect()`.

А если блок `flow` начинает создавать новые корутины, то в них он и запустит метод `emit()`, а следовательно и блок `collect`. И в этом заключается основная проблема. Мы (как получатель) ожидаем, что блок `collect` выполнится в нашей текущей корутине (где мы запускаем метод `collect()`), а он выполнится неизвестно в какой корутине, которую запустил блок `flow`.

Самый очевидный пример - это когда мы хотим получать данные из `Flow` в `Main` потоке. Метод `collect()` мы вызываем в корутине с `Main` диспетчером. В блоке `collect` мы получаемые данные выводим на экран, и можем это сделать только в `Main` потоке. Т.е. запуская метод `collect()` в корутине с `Main` потоком мы ожидаем, что и блок `collect` будет выполнен в этой же корутине. А если блок `flow` вызовет `emit()` (а следовательно и блок `collect`) из какой-то своей корутины (например, с `IO` диспетчером), то наш код по выводу данных на экран не сработает, т.к. будет выполнен не в `Main` потоке.

Более подробно об этом можно прочесть в [статье](#) (англ) Романа Елизарова.

В общем, основной вывод - в блоке `flow` нельзя вызывать `emit` из корутин. Для решения этой проблемы есть специальный билдер - `channelFlow`.

## channelFlow

Чтобы лучше понять, что именно делает `channelFlow`, давайте сами сделаем его упрощенную версию.

Основная проблема, с которой мы столкнулись в прошлом примере - это то, что в блоке `flow` нельзя вызывать `emit` из другой корутины. Т.е. можно запустить другую корутину и создать там данные, а вот отправить их оттуда получателю методом `emit` нельзя, т.к., тем самым, мы меняем корутину, в которой будет запущен код получателя (блок `collect`). Это значит, что нам надо вызов `emit` вернуть в нашу текущую корутину.

Т.е. план такой:

- 1) в нашей текущей корутине в блоке `flow` создаем новую корутину
  - 2) в новой корутине создаем данные
  - 3) передаем эти данные из новой корутины в текущую корутину
  - 4) в текущей корутине запускаем метод `emit()` чтобы отправить данные получателю
- Профит!

Основная сложность в этом плане - пункт 3. Нам надо передать данные из одной корутины в другую. И это может быть не одно значение, а последовательность значений, т.к. мы работаем с `Flow`. Т.е. простым `async-await` тут не отделаться.

И тут мы вспоминаем, что для безопасной и удобной передачи последовательности данных из одной корутины в другую отлично подходят каналы!

В уроке про каналы мы рассматривали билдер `produce`. Он создает связку: корутина + канал.

```

1 | val channel = produce<Int> {
2 |     // send(...)
3 | }

```

Билдер `produce` возвращает нам канал. А блок кода, который мы передаем в `produce`, будет выполнен в новой корутине. Там мы сможем вызывать метод `send` для отправки данных в канал.

И это как раз то, что нам нужно чтобы из текущей корутины запустить новую корутину для создания данных. А чтобы получать данные из канала обратно в текущую корутину, можно использовать `consumeEach`:

```

1 | channel.consumeEach {
2 |     // ...
3 | }

```

Давайте вставим эту конструкцию в `flow` билдер.

```

1 | flow {
2 |     coroutineScope {
3 |         val channel = produce<Int> {
4 |             // ...
5 |         }
6 |         channel.consumeEach {
7 |             // ...
8 |         }
9 |     }
10 | }

```

*Несмотря на то, что `coroutineScope` под капотом создает новую корутину, `Flow` считает, что все происходящее внутри `coroutineScope` происходит в текущей корутине. А значит, мы можем вызвать там метод `emit`. Это учитывается в подкапотных проверках и не приведет к ошибке.*

Итак, `produce` создает нам новую корутину. Туда мы поместим код создания данных. А методом `send` будем отправлять их в канал.

```

1 | flow {
2 |     coroutineScope {
3 |         val channel = produce<Int> {
4 |             launch {
5 |                 delay(1000)
6 |                 send(1)
7 |             }
8 |             launch {
9 |                 delay(1000)
10 |                send(2)
11 |            }
12 |            launch {
13 |                delay(1000)
14 |                send(3)
15 |            }
16 |        }
17 |        channel.consumeEach {
18 |            // ...
19 |        }
20 |    }
21 | }

```

В `produce` создаем три новые корутины, чтобы создавать данные параллельно. Если вам не нужна такая параллельность, то можно все сделать прямо в `produce` без создания новых дополнительных корутин.

Созданные данные мы отправляем в канал методом `send`. Получать их будем в `consumeEach`. Напомню, что `consumeEach` является `suspend` функцией и выполняется в текущей корутине, не создавая никаких новых корутин. А, значит, мы можем в этой функции вызвать метод `emit()`, чтобы перенаправить эти данные получателю `Flow`:

```

1 | flow {
2 |     coroutineScope {
3 |
4 |         val channel = produce<Int> {
5 |             launch {
6 |                 delay(1000)
7 |                 send(1)
8 |             }
9 |             launch {
10 |                delay(1000)
11 |                send(2)
12 |            }
13 |            launch {
14 |                delay(1000)
15 |                send(3)
16 |            }
17 |        }
18 |        channel.consumeEach {
19 |            emit(it)
20 |        }
21 |    }
22 | }
23 |

```

В итоге мы получили `Flow`, внутри которого будут созданы корутин и канал. Когда мы запустим его методом `collect()`, данные будут создаваться в отдельной корутине и с помощью канала вернуться в нашу текущую корутину.

Именно так и работает `channelFlow`. Пример вызова:

```

1 | val flow = channelFlow {
2 |     launch {
3 |         delay(1000)
4 |         send(1)
5 |     }
6 |     launch {
7 |         delay(1000)
8 |         send(2)
9 |     }
10 |    launch {
11 |        delay(1000)
12 |        send(3)
13 |    }
14 | }

```

Все лишнее спрятано под капотом, нам остается только написать блок кода, который будет создавать данные и отправить их методом `send`. Т.е. тот же код, что мы использовали в `produce` в нашем примере.

В билдеры `launch` мы можем передавать контекст (диспетчер, хэндлер и т.п.), который нам нужен.

Рассмотрим еще несколько операторов, которые делают похожие преобразования, используя каналы.

## flowOn

Оператор `channelFlow` позволял нам самим создать `Flow`, чтобы выполнять его код в корутине с нужным нам контекстом. Но что, если нам дали уже готовый `Flow`. Как нам попросить выполнить его блок `flow` в отдельной корутине?

Если бы нам пришлось делать это самим, то мы бы сделали так:

```

1 | flow {
2 |     coroutineScope {
3 |         val channel = produce<Int>(neededContext) {
4 |             flow.collect {
5 |                 send(it)
6 |             }
7 |         }
8 |         channel.consumeEach {
9 |             emit(it)
10 |        }
11 |    }
12 | }

```

Снова используем `produce`. При его вызове указываем контекст, который нужен - `neededContext`. Внутри мы методом `collect` запускаем `Flow`, который нам дали. Он выполнится в корутине `produce` и оттуда отправит данные в канал методом `send`. А мы их получим в `consumeEach` и методом `emit` отправим получателю.

Т.е. разница с прошлым примером в том, что мы в новой корутине не сами создаем данные, а получаем их из потока `flow`, который нам дали. И при получении сразу отправляем их в канал.

Именно так и работает оператор `flowOn`. Мы используем его, когда хотим сообщить `Flow`, какой контекст ему использовать для кода создания данных:

```

1 | val ioFlow = flow.flowOn(Dispatchers.IO)

```

Вызов `ioFlow.collect()` приведет к тому, что `flow` начнет работу в `IO` потоке. А все данные мы будем получать в текущей корутине.

В прошлом уроке мы рассматривали `Intermediate` операторы. Если помните, каждый из них создает `Flow` обертку и мы можем комбинировать их. Оператор `flowOn` также создает `Flow` обертку и может быть встроен в цепочку операторов в любом месте.

Пример:

```

1 | flow {
2 |     // ... IO thread
3 | }
4 | .map {
5 |     // ... IO thread
6 | }
7 | .flowOn(Dispatchers.IO)
8 | .onEach {
9 |     // ... Main thread
10 | }
11 | .flowOn(Dispatchers.Main)

```

Здесь билдер `flow` и оператор `map` будут выполнены в `IO` потоке. А оператор `onEach` - в `Main` потоке. Т.е. оператор `flowOn` влияет на все предыдущие операторы в цепочке, пока не встретит другой `flowOn`.

Почему `flowOn` влияет на все предыдущие операторы, а не только на тот, к которому он был применен? Рассмотрим на примере выше.

У нас есть цепочка `flow > map > flowOn`. `flowOn` создает `Flow` обертку. Эта обертка запустит `map` в `IO` потоке. `map` в свою очередь также является оберткой, которая будет запускать `flow`. Но в отличие от `flowOn`, `map` - это простая обертка, которая лишь выполняет преобразование данных и не создает новых корутин. Поэтому `map` просто запустит `flow` в той же `IO` корутине, где сам `map` и выполняется. В итоге `flow` выполнится в той корутине с `IO` диспетчером, которую запустил `flowOn`.

В итоге все данные будут получены в той корутине, в которой мы вызываем метод `collect()`. И для этого нам не надо использовать никакие операторы.

## buffer

Оператор `buffer` дает нам функционал аналогичный буферу, который мы рассматривали в уроке про каналы. Т.е. если отправитель шлет данные слишком быстро и получатель не успевает их обработать, то данные будут скапливаться в буфере.

Под капотом абсолютно тот же механизм, что и у `flowOn`.

```
1 | flow {
2 |     coroutineScope {
3 |         val channel = produce<Int>(capacity) {
4 |             flow.collect {
5 |                 send(it)
6 |             }
7 |         }
8 |         channel.consumeEach {
9 |             emit(it)
10 |        }
11 |    }
12 | }
```

Отличаются только цели использования `produce`. `flowOn` его использовал, чтобы перейти в другую корутину с указанным контекстом, а `buffer` использует, чтобы перейти на использование канала с указанным режимом/размером буфера (`capacity`).

*Обратите внимание на важный момент. Оператор `buffer`, как и оператор `flowOn`, запустит новую корутину для запуска тех `Flow`, которые находятся перед ним в цепочке операторов. Потому что он использует `produce`. Но в отличие от `flowOn` он не будет менять контекст. Т.е. корутина будет новая, но диспетчер останется прежним.*

Использование буфера выглядит так:

```
1 | val flow = flow {
2 |     // ...
3 | }.buffer(5)
```

Операторы `buffer` и `flowOn` можно комбинировать.

```
1 | val flow = flow {
2 |     // ...
3 | }
4 | .buffer(5)
5 | .flowOn(Dispatchers.IO)
```

При этом оператор `flowOn` не будет создавать новый `produce`. Он увидит, что прямо перед ним находится `buffer` и просто добавит свой контекст в вызов `produce` внутри `buffer`.

## produceIn

Оператор `produceIn` тоже использует `produce`, чтобы отправить данный ему `Flow` работать в новую корутину с каналом. Но при этом он сам не будет получать данные из канала. Он вернет этот канал нам.

Т.е. под капотом это выглядит примерно так:

```
1 | val channel = scope.produce<Int> {
2 |     flow.collect {
3 |         send(it)
4 |     }
5 | }
```

В `produce` запускается переданный ему `Flow`, и все его данные отправляются в канал. И никакая `Flow` обертка не создается. Вместо этого мы получаем канал.

Пример использования:

```
1 | val channel = flow {
2 |     // emit
3 | }
4 | .produceIn(scope)
```

Таким образом `Flow` конвертируется в канал.

Обратите внимание на три важных момента.

В `produceIn` надо передавать `scope`, который будет использован для запуска `produce`.

Получившийся канал (в отличие от обычного `Flow`) не является `Cold`. Т.е. он начнет работу сразу же, как мы его создадим.

Если нам надо указать свой контекст или параметры буфера, мы используем операторы `flowOn` и `buffer`:

```
1 val channel = flow {  
2     // emit  
3 }  
4 .buffer(5)  
5 .flowOn(Dispatchers.IO)  
6 .produceIn(scope)
```

